

## Lista zadań nr 10

### Zadanie 1. (2 pkt)

Napisz funkcję, która dla danego programu maszyny wirtualnej oblicza maksymalną głębokość stosu potrzebną do jego wykonania.

### Zadanie 2. (2 pkt)

Dla języka z let-wyrażeniami napisz funkcję odwrotną do kompilacji, która dla danego kodu maszyny wirtualnej odtwarza odpowiadające mu wyrażenie.

### Zadanie 3. (2 pkt)

Kompilator z wykładu jest nieefektywny, ponieważ używa konkatenacji list do łączenia kodu, co może prowadzić do kwadratowego czasu kompilacji. By temu zaradzić, dodaj do funkcji kompilującej dodatkowy argument — listę instrukcji które należy dołączyć na końcu wygenerowanego kodu. Nie używaj odwracania list.

### Zadanie 4. (3 pkt)

Zauważ, że maszyna wirtualna w ostatniej wersji kompilatora z wykładu (`calc_6809`) zawiera następujące instrukcje:

- `MPopAcc` — zdejmuję ze stosu wartość i umieszcza ją w akumulatorze,
- `MAlloc  $n$`  — alokuje na stercie blok pamięci o rozmiarze  $n$  i umieszcza jego adres w akumulatorze,
- `MGet  $n$`  — odczytuje  $n$ -tą komórkę z bloku pamięci, którego adres znajduje się w akumulatorze, a następnie umieszcza tę wartość w akumulatorze,

- `MSet  $n$`  — zapisuje wartość z akumulatora do  $n$ -tej komórki bloku pamięci, którego adres znajduje się na stosie. Adres nie jest usuwany ze stosu, dzięki czemu można go używać przy kolejnych instrukcjach `MSet`. Na przykład, poniższy ciąg instrukcji tworzy parę wartości 13 i 42.

```
MAlloc 2
MPush
MConst 13
MSet 0
MConst 42
MSet 1
MPopAcc
```

Użyj powyższych instrukcji, by rozszerzyć implementację kompilatora o obsługę par (tworzenie pary oraz operacje `fst` i `snd`). Oczywiście rozszerz lekser i parser o nowe konstrukcje składniowe.

**Uwaga:** kod umieszczony po wykładzie zawierał drobny błąd przy tłumaczeniu instrukcji maszyny wirtualnej na kod asemblera 6809. Poprawiona wersja jest już dostępna w SKOSie.

### Zadanie 5. (1 pkt)

Rozszerz implementację ewaluatora maszyny wirtualnej o instrukcje z poprzedniego zadania. Do implementacji bloków pamięci użyj modułu `Array`.

### Zadanie 6. (1 pkt)

Powtórz poprzednie zadanie, ale używając wyłącznie trwałych struktur danych. W tym celu dodaj do ewaluatora dodatkowy argument reprezentujący stan sterty.

### Zadanie 7. (1 pkt)

Typy w języku z let wyrażeniami oraz parami można opisać następującym typem danych.

```
type tp =
  | TInt
  | TPair of tp * tp
```

Napisz funkcję, która dla danego wyrażenia zwraca jego typ lub sygnalizuje błąd, jeśli wyrażenie jest niepoprawnie typowane (na przykład, jeśli próbujemy dodać do siebie dwie pary). Zauważ, że funkcja ta znów będzie miała kształt ewaluatora.